

Arduino IDE (Integrated Development Environment)

- One stop solution for programming Arduino boards.
- Any board provider (even the non-Arduino board) can make their configuration files through Arduino Board Manager.
- This is why we can use Arduino IDE for programming ESP32, which is not an Arduino board but a popular microcontroller board.

Arduino IDE Features

1. **Code Editor** - Syntax highlighting, auto-completion
2. **Compiler** - GCC-based compilation for microcontrollers
3. **Upload Tool** - Flashes program to Arduino memory
4. **Serial Monitor** - Debug console (like `stdout`)
5. **Library Manager** - Package manager for Arduino libraries

What is needed for programming embedded systems?

1. Compiler: mainly C/C++ compiler to convert source code to object code.

Object code: is a binary code that translates high-level language to machine language in a specific file format (e.g., ELF, HEX, BIN) for linking and loading.

Ask LLM

- What is the object code format for your OS?
- What is the library? What's the difference between the object file and the library file?
- How to combine (link) multiple object files into a single executable file?

2. Linker: to combine multiple object files into a single executable file.

For OS based systems, the C/C++ compiler generates an executable file (e.g., ELF, EXE, etc.) that can run on the same OS.

For embedded systems, the C/C++ compiler generates a binary file for the Arduino board.

Ask LLM

- What is the cross-compiler?
- How come I can use my PC/Mac for generating code for Arduino board?
- How to send the binary file for the Arduino board in my PC/Mac down to the Arduino board?
- What is the Intel HEX format?

3. Uploader: to send the binary file to the Arduino board via USB or other communication interfaces.

We need to translate the binary file to a format that the Arduino board can understand (e.g., Intel HEX format) and send it via USB or other communication interfaces.

We use the `objcopy` tool to convert the binary file to Intel HEX format.

Compilation Process:

```
Arduino Code (.ino)
    ↓ (Preprocessor)
C++ Code
    ↓ (GCC Compiler + Linker)
Machine Code
    ↓ (Upload)
Arduino Flash Memory
```

We need to use Arduino bootloader to receive the Intel HEX file and program it to the flash memory of the ATmega328P microcontroller on the Arduino board.

Arduino Bootloader

Arduino boards have a bootloader pre-installed in the microcontroller's flash memory.

1. It is a small program that runs when the board is powered on or reset.
2. It listens for incoming data on the serial port (USB) for a short period.
3. If it receives a valid program, it writes it to the microcontroller's flash memory.

4. If no valid program is received, it jumps to the existing program in flash memory.
5. The Arduino Bootloader copies the Intel HEX file to the microcontroller's flash memory.
6. The Arduino CPU executes the program from flash.

The Connection between PC/Mac and Arduino Board

- Arduino IDE should know all the details of the Arduino board to generate the correct binary file and send it to the board.
- It also needs to know the correct communication port to send the binary file to the board.

The Hardware I/O & Memory Map

Arduino Uno R3

ATmega328P

- 32KB Flash (Program)
- 2KB SRAM (Variables)
- 1KB EEPROM (Persistent)
- 16MHz Clock

Digital Pins: 0–13 (6 PWM)

Analog Pins: A0–A5

Power: 5V, 3.3V, GND

- **Programming Analogy:** Think of pins as **API endpoints** - each pin has specific capabilities and protocols.
- Instead of using a number of I/O ports, we can use a programming symbols (e.g., `LED_BUILTIN`) to refer to the specific pin for the built-in LED.
- However, this is only valid for well known boards (e.g., Arduino Uno, Nano, etc.) that are supported by the Arduino IDE.

Summary of Arduino IDE

Using Arduino IDE:

- We don't need to configure the hardware details of the Arduino board.
- We don't need to worry about setting up the toolchain (compiler, linker, uploader, etc.)

Discuss with LLM

Go back to the initial questions:

1. Why is an Arudino board so popular among beginners in embedded systems?
 - What problems did it solve to be so successful?
2. Why is the Aarduino IDE is so popular that many boards, including ESP32, support it?
 - What problems did it solve to be so successful?

1. What are your answers?
2. What are LLM answers?
3. Ask more questions to deepen your understanding.
4. Finally, organize your thoughts and ideas in your own words to your "2nd brain"